ONSITE PROJECT PRESENTATION · NOVEMBER 18, 2026

Secure Sensor Link

A receiver that decodes, authenticates and time-syncs camera frames arriving over a hostile link

Andy Chan

FOLLOWS THE DESIGN DOC

Contents

01	The project	slide 3
02	Threat model	slide 10
03	Security assurance	slide 17
04	Frame check order	slide 31
05	Design decisions	slide 34
06	Implementation and live demo	slide 40
07	Test coverage	slide 49
08	Performance	slide 51
09	Future enhancements	slide 52
10	AI usage	slide 54

THE PROJECT

A receiver for camera frames from a hostile link

Two cameras
Sign every frame with HMAC-SHA256

→

Hostile link
An attacker can read, drop, delay, replay, modify and inject

→

The receiver
Decodes, authenticates, time-synchronizes

→

Paired output
One stream of time-matched frame pairs

TRUSTED SIDE One JSON command per line on stdin, from the node's own software. Frames arrive as hex inside receive commands.

Stage 1
Authenticated frames, replay protection, timestamp correction, pairing

Stage 2
Key rotation and revocation

Stage 3
Replay protection that survives a restart

THE PROJECT

A frame on the wire

version 1 B	camera 1 B	key id 1 B, v2 only	seq 4 B	timestamp 8 B	payload length 2 B	payload: a tree 0 to 65,535 B	tag 32 B

The tag is HMAC-SHA256, with the camera’s key, over every byte before it

Header

- Version 1 has no key id and uses key 0
- seq: unsigned 32-bit, up by one per frame
- timestamp: signed 64-bit, in milliseconds
- Largest frame: 65,584 bytes

Payload

- A binary tree in preorder: 0x00 for no node, 0x01 for a node
- A node: a 4-byte signed value, then its left and right subtrees
- Reported back in level order

THE PROJECT

Six commands, one JSON line each

Command	What it does
init	Sets the timing limits, max_pending, persist_interval, and each camera's key 0
receive	Checks one frame, given as hex; returns it decoded, plus any pairs it releases
get_state	Each camera's last seq and timestamp, waiting frames and key statuses
add_key	Provisions a new key id for a camera
revoke_key	Revokes a camera's key: frames under it are refused from then on
restart	A power loss: memory is wiped and rebuilt from storage

Every line gets exactly one response line, with a status of ok or error.

THE PROJECT

Correcting timestamps

```
expected = T + (seq - S) × period_ms
```

> T and S: the camera’s last accepted timestamp and seq

> Off by more than max_jitter_ms? The expected value replaces it

> A camera’s first frame keeps its timestamp

Example: cam2’s clock glitches and sends 9999; it becomes the expected 1055

THE PROJECT

Pairing frames from the two cameras

- Each camera has a queue of accepted frames
- The two oldest within pair_window_ms: one pair, stamped with the floor of their average
- Otherwise the older one goes out unpaired
- A queue past max_pending sends its oldest out unpaired

THE PROJECT

Key rotation and revocation

- Each camera has key ids 0 to 255
- add_key provisions a key; the first frame signed with a higher key retires every lower one
- revoke_key: frames under that key are refused for good
- Each key is active, retired or revoked

THE PROJECT

Restarts

> restart wipes memory; keys and replay protection come back from storage

> **Spec rule:** never accept a frame it would have rejected without the restart

> **Spec rule:** lose at most persist_interval frames past the last accepted one

> Storage wears out, so floor writes per camera are capped

THREAT MODEL

The attacker

> Can observe, drop, delay, duplicate, reorder, modify and inject frames

> Reaches the receiver only through a frame’s bytes, as hex inside a receive command: can’t break out of the JSON, add a command or see responses, which trusted processes on the node handle

> Doesn’t know the camera keys, at least the ones still active

THREAT MODEL

What we assume

> Cameras are trusted, but clocks occasionally glitch; seq goes up by one per frame, even across a camera’s reboots

> Commands on stdin are trusted; key management arrives over a trusted channel

> Node memory and storage are trusted: keys encrypted or in a secure element; writes atomic, durable, rollback-protected

> Camera clocks agree within pair_window_ms; otherwise frames are released unpaired

THREAT MODEL

Security goals: S1 to S4

Goal	Description
S1 Authenticity	Only accept frames with a valid tag under an active key
S2 No replay	Don’t accept duplicates of genuine frames, even across restarts
S3 Key lifecycle	Retired and revoked keys never verify again, and key IDs only move forward
S4 Key secrecy	Key material is not inadvertently leaked; defense in depth against memory later leaking (e.g. core dump)

THREAT MODEL

Security goals: S5 to S8

Goal	Description
S5 Robustness	The receiver must never crash, hang, or let its memory grow without bound, whatever input it receives
S6 Output integrity	Only accepted frames reach the paired output, and every timestamp is computed exactly as the spec requires, with no overflow or wraparound
S7 Atomic state	A rejected or failed command keeps the receiver in a consistent state, and storage is never behind memory
S8 Restart safety	A restart never lets in a frame that an unrestarted receiver would reject as a replay or under an old key, and loses at most persist_interval genuine frames per camera

THREAT MODEL

Out of scope

> Hardware-level side channels and fault injection, which need either physical access to the node (power and EM analysis, glitching, laser, EM pulse) or the attacker's own code running on it (Spectre-style speculative execution, Rowhammer)

> Confidentiality of the frame data

> A compromised key that's still active

THREAT MODEL

Known limitations (1 of 2)

- A failed memory allocation aborts the process, although allocation is kept to a minimum
- A bad first timestamp puts every later frame from that camera on a wrong timeline
- A stolen key can push seq to its maximum, rejecting the camera’s later genuine frames even after revocation; there’s no re-provisioning command yet
- A stolen key can also put genuine frames on the attacker’s timeline, so they stop pairing; a restart after revoking the key resets the timeline
- A restart causes up to persist_interval frames per camera to be lost
- A fault mid-commit rebuilds memory from storage, as a restart does, with the same loss

THREAT MODEL

Known limitations (2 of 2)

- With a wide pair window, one dropped frame silently shifts every later pair by one period
- Revoking a key can drop up to max_pending queued frames verified with it
- Revoking a key doesn’t recall frames already emitted
- Key text sits briefly in the operating system’s pipe buffer
- Decoding a key from hex isn’t constant time, but key text only arrives over a trusted channel

SECURITY ASSURANCE

Assurance layers, strongest first

Layer	Explanation
By construction	Rust with no unsafe, Clippy lints and release overflow checks; the compiler and CI reject regressions
By design	RawFrame becomes VerifiedFrame only by verification; a read-only view of state until all checks pass; key copies that erase themselves; a Config only built with valid values
Spec conformance	Tested against all the examples in the project description
Design decisions	One test per rule and decision
Independent implementation	Tested against a simplified reference model over random command sequences
Coverage-guided fuzzing	Three targets, each asserting invariants on every input

SECURITY ASSURANCE

Assurance layers, continued

Layer	Explanation
Running process	Memory scan for keys, and memory growth monitoring
Test quality	Mutation testing and coverage analysis
Supply chain	Only libraries the spec allows; cargo-deny checks advisories, licenses and sources
Review	Security review by both AI and a human
CI	On every push: tests in debug and release on Linux and macOS; formatting, lints and docs; coverage; each fuzzer for 60 seconds; cargo-deny

ENFORCING THE SECURITY GOALS

S1 Authenticity

Goal: Only accept frames with a valid tag under an active key.

> The tag covers every byte of the frame, and is checked in constant time with RustCrypto's verify_slice

> Key status is checked before verification, so a retired or revoked key is never used to verify

> A RawFrame only becomes a VerifiedFrame through verification: the code can't read its seq, timestamp or payload before

Note: keys must stay secret. Constant time is RustCrypto's property, not verified here: defense in depth, since attackers can't measure timing directly.

Checked by: the order-of-checks tests, forged frames in the reference model, and the receiver fuzzer

S2 No replay

Goal: Don’t accept duplicates of genuine frames, even across restarts.

> Only strictly increasing sequence numbers are accepted

> The restart floor is saved to storage before accepting, so a restart can’t forget an accepted seq

> Example: after seq 6, a replay of seq 5 or 6 is rejected. With a saved floor of 101, after a restart seq 101 is rejected and seq 102 is accepted

Checked by: the rules tests, the reference model, the restart twin test, and the receiver fuzzer across restarts

S3 Key lifecycle

Goal: Retired and revoked keys never verify again, and key IDs only move forward.

- Key status is checked before frame verification, and revoked or retired keys are erased immediately
- add_key refuses IDs below the current key, and retired or revoked IDs
- A frame can only move its camera to a higher, active key
- After a restart, a frame refused only by the restart floor still moves its camera’s key forward
- Every key status change is saved to storage before it takes effect

Note: a stolen key that’s still active is out of scope until revoked.

Checked by: the key rotation and revocation tests, the restart twin test, and the receiver fuzzer

S4 Key secrecy: the input path

Goal: Key material is not inadvertently leaked; defense in depth against memory later leaking (e.g. core dump).

- The receiver reads stdin through its own 64 KiB buffer instead of the standard one, and erases it as it’s consumed
- The line buffer is zeroed after every command
- A key’s hex text is decoded right from the line into the key’s own memory, so no other copies are made
- A key containing any escape sequence is rejected, so the JSON library never makes a decoded copy

Checked by: the memory scan on Linux, and the rules tests for escapes

S4 Key secrecy: memory and output

Goal: Key material is not inadvertently leaked; defense in depth against memory later leaking (e.g. core dump).

> Keys erase themselves when dropped, and the hmac library wipes its own internal state when its working object is dropped

> A printed key shows as Key(redacted), and error messages never repeat a string from the input

> After each command, 64 KiB of zeroes are written over the stack, clearing temporary key copies left there by the JSON and hmac libraries

Note: the operating system’s pipe buffer can still hold a temporary copy of the key text.

Checked by: the memory scan, which counts key copies in the live process; a release build also catches a missing stack scrub

ENFORCING THE SECURITY GOALS

S5 Robustness

Goal: Never crash, hang, or let memory grow without bound, whatever the input.

- Lint options rule out common sources of crashes; build settings turn on overflow checks
- A panic safety net catches unexpected panics and returns “internal error” instead
- Large buffers are allocated once and reused for every command
- Strict caps on lines, trees and queues
- No recursion, threads or locks; every loop makes progress on bounded input
- The worst case, the largest frame with a full tree releasing a full queue, is still fixed work

Note: the receiver can still wait on stdin and stdout, which is expected.

Checked by: all three fuzz targets, the soak test, the size-limit tests, and the end-to-end tests

S6 Output integrity

Goal: Only accepted frames reach the paired output, and every timestamp is computed exactly as the spec requires, with no overflow or wraparound.

- Queues only hold accepted frames
- Revoking a key drops the queued frames verified with that key ID
- Timestamp calculations use 128-bit checked arithmetic

Note: frames already emitted can’t be recalled.

Checked by: the reference model’s exact responses, the spec examples, and the tests at the arithmetic extremes

ENFORCING THE SECURITY GOALS

S7 Atomic state

Goal: A rejected or failed command keeps the receiver consistent, and storage is never behind memory.

1. Plan
Every check, on a read-only view

→

2. Mark
Set the commit-in-progress flag

→

3. Save
Write storage, atomically

→

4. Apply
Update memory

→

5. Done
Clear the flag and reply

> Everything is verified first, with a read-only reference to the receiver's state

> The storage write, assumed atomic, is done before the change in memory

> A panic during the storage write, or a failure during the memory change, replies "internal error" and leaves the flag set; before the next command, memory is rebuilt from storage, as a restart would

Checked by: the safety-net tests, the failing-store tests, and the restart twin test

S8 Restart safety

Goal: A restart never lets in a frame an unrestarted receiver would reject as a replay or under an old key, and loses at most persist_interval frames per camera.

> Before accepting a frame, save floor = seq + persist_interval if the existing floor is below the new seq. After a restart, seqs at or below the floor are rejected. Writes stay within the spec's budget

> After a restart, a frame refused only by the floor, if otherwise valid, still retires old keys, so someone holding an old key can't get a frame accepted

Checked by: the restart twin test, the write-budget tests, and the receiver fuzzer across restarts

SECURITY ASSURANCE

Defense against attacks (1 of 3)

Attack	Defense
Forge or alter a frame without a key	HMAC-SHA256 over the whole frame, with a constant-time check
Replay a captured frame	Sequence number check
Replay after a restart	A seq at or below the saved floor is rejected
Wear out the storage by flooding frames	Verification first; floor writes stay within the spec’s budget
Reorder frames	Signed seq; anything at or below the last accepted is stale
Delay frames	Delayed frames keep their signed timestamps, so the consumer can judge their age
Drop frames	Bounded queues; frames released unpaired when no partner can come; with a wide pair window, one dropped frame shifts every later pair

SECURITY ASSURANCE

Defense against attacks (2 of 3)

Attack	Defense
Use a retired key captured before a rotation	Key status checked first; keys only move forward; below the floor, the key still moves
Use a revoked key	Revocation is permanent, and its queued frames are dropped; last_seq and the timeline stay
Send malformed frames	Strict parsing, no recursion, no panics
Exhaust memory or time	Buffers allocated once; caps on lines, trees and queues; bounded work
Force an overflow	Checked arithmetic, 128-bit timestamps and overflow checks
Time the tag check	Constant-time comparison, as defense in depth

SECURITY ASSURANCE

Defense against attacks (3 of 3)

Attack	Defense
Probe error messages	Messages do not reveal sensitive information
Get a misplaced key echoed back	Errors never repeat input; keys print as Key(redacted)
Leave the state half-changed	Storage first, then memory; after a half-done change, memory is rebuilt from storage
Recover keys from memory or dumps	Keys, input buffers and the stack are erased
Get a forged or revoked-key frame into the output	Only accepted frames are queued; revocation drops queued frames
Time SHA-256 through the cache	SHA-256 has no tables indexed by secret data

FRAME CHECK ORDER

Nine steps, with the tag as the trust boundary

Steps 1–4: untrusted bytes · **Step 5:** the trust boundary · **Steps 6–9:** a VerifiedFrame

1. Hex length
Valid hex, even length, at most 2 × 65,584

2. Version
First byte exists, and is 1 or 2

3. Layout
Known camera; exact frame length

4. Key status
Provisioned, not revoked, not retired

5. Tag
HMAC-SHA256 over everything before it, in constant time

6. Sequence number
Above the last; after a restart, above the floor

7. Timestamp
Not negative

8. Tree
Valid, at most 1,024 nodes and 64 levels

9. Accept
Correct the timestamp, plan pairing, commit

FRAME CHECK ORDER

Why each step sits where it does

Step	Why here
1. Hex length	A cheap check to quickly reject oversized frames
2. Version	The two versions have different header layouts
3. Layout	Nothing is trusted before the tag: verify only what the tag check needs
4. Key status	A retired or revoked key is never even tried
5. Tag	The major trust boundary: only now is the frame a VerifiedFrame
6. Sequence number	Forged frames are reported as authentication failures, not stale
7. Timestamp	A cheap check for spec conformance
8. Tree	The most complex parser runs last, only on authenticated data
9. Accept	The final step, and the only one that changes storage and memory

FRAME CHECK ORDER

Considerations

Key status (4) before the tag (5)

Revoked and retired keys are erased, so their tags can’t be checked.

Trade-off: anyone who sees responses learns a key’s status, as in the spec’s example.

Sequence number (6) after the tag (5)

A forged frame is reported as an authentication failure, not stale or replayed, so whatever reads the responses can detect injection.

Trade-off: a bad seq still costs a tag check, but an attacker can force one anyway with a fresh seq.

Step 6 special case

For a frame below the restart floor, steps 7 and 8 still run. If it’s valid, it still moves its camera to a newer key.

Either way, the frame is reported as stale.

DESIGN DECISIONS · QUESTION 1 OF 5

Tree limits

Accept trees beyond 1,024 nodes or 64 levels?

DECISION

Reject trees with over 1,024 nodes or 64 levels.

RATIONALE

A bounded worst case the spec still allows. The limits are named constants, so changing them is a one-line change.

DESIGN DECISIONS · QUESTION 2 OF 5

Valid tag, invalid payload

Should the frame still count for replay protection?

DECISION

Rejected, without counting for replay protection.

RATIONALE

Keeps the spec’s restart rules, where only accepted frames count, and fits plan-then-commit. An invalid payload is an error case, not one to optimize.

DESIGN DECISIONS · QUESTION 3 OF 5

Wide pair window

What if one frame could pair with two from the other camera?

DECISION

Pair it with the oldest first.

RATIONALE

Follows the spec’s pairing steps as written, with no look-ahead. But if 2 × pair_window_ms ≥ period_ms – max_jitter_ms, one dropped frame silently shifts every later pair by one period; below that, it can’t happen.

DESIGN DECISIONS · QUESTION 4 OF 5

add_key with an existing, revoked or lower ID

What should add_key do?

DECISION

Revoked, below the current key, or different bytes: an error. Same ID and bytes: success, no change.

RATIONALE

Never revive or change a key, and keep retries idempotent, e.g. when a power cut loses the reply.

DESIGN DECISIONS · QUESTION 5 OF 5

Queued frames when their key is revoked

What happens to frames waiting in a queue?

DECISION

Dropped without being emitted; last_seq and the timeline stay as they are; frames under a retired key stay.

RATIONALE

Removes possibly forged frames, at the cost of up to max_pending frames. Rolling back last_seq goes against the spec’s rules, and could let replays of already-reported frames in.

DESIGN DECISIONS

Other questions: the restart floor



Floor and write budget

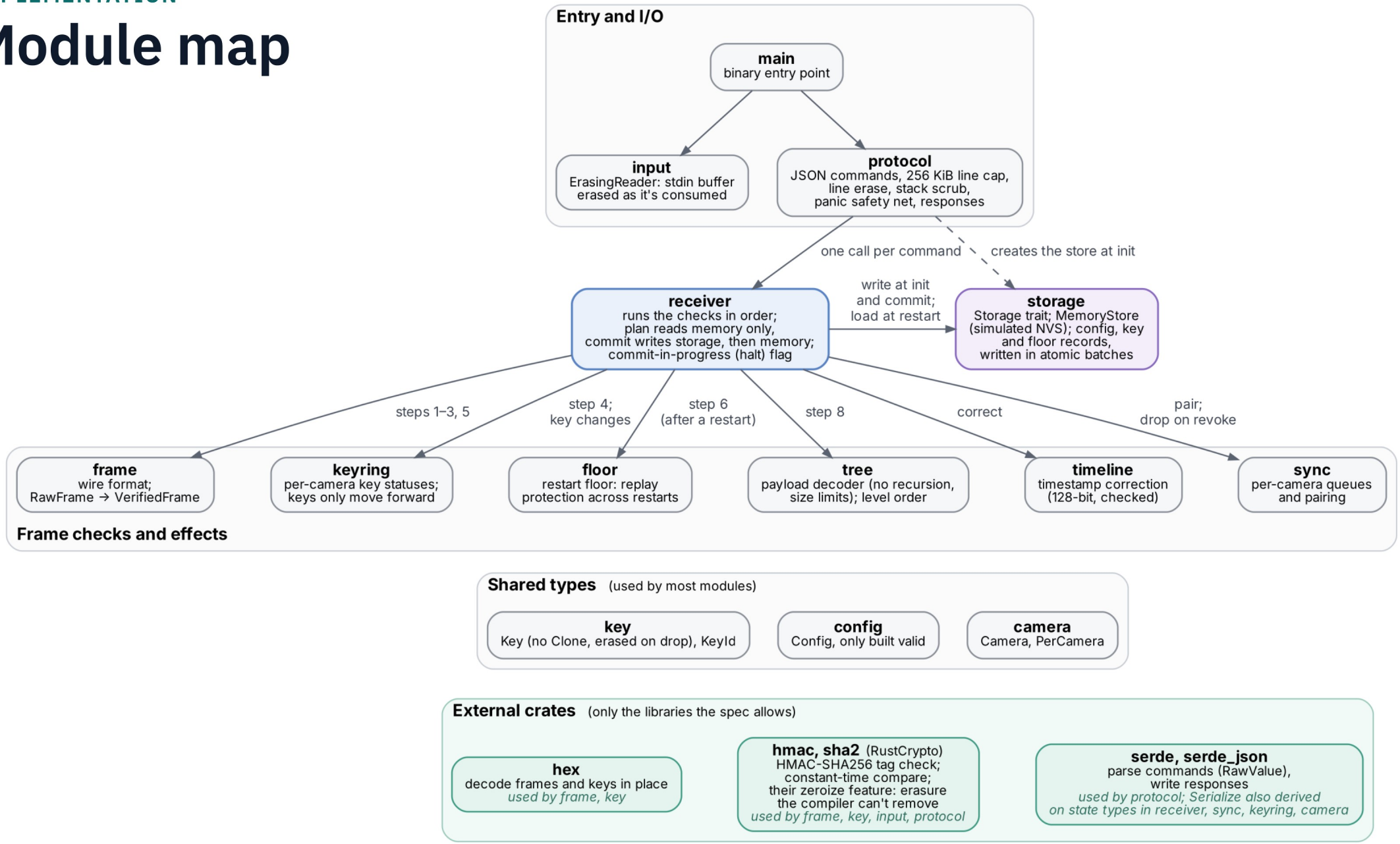
Save floor = seq + persist_interval before accepting past it, then write nothing until a seq passes it. Writes stay within 1 + (S_max − S_min) / persist_interval.

One exception: the key still moves

A frame rejected only by the floor, but otherwise valid, still moves its camera to its newer key, so someone holding an old key can’t get a frame accepted.

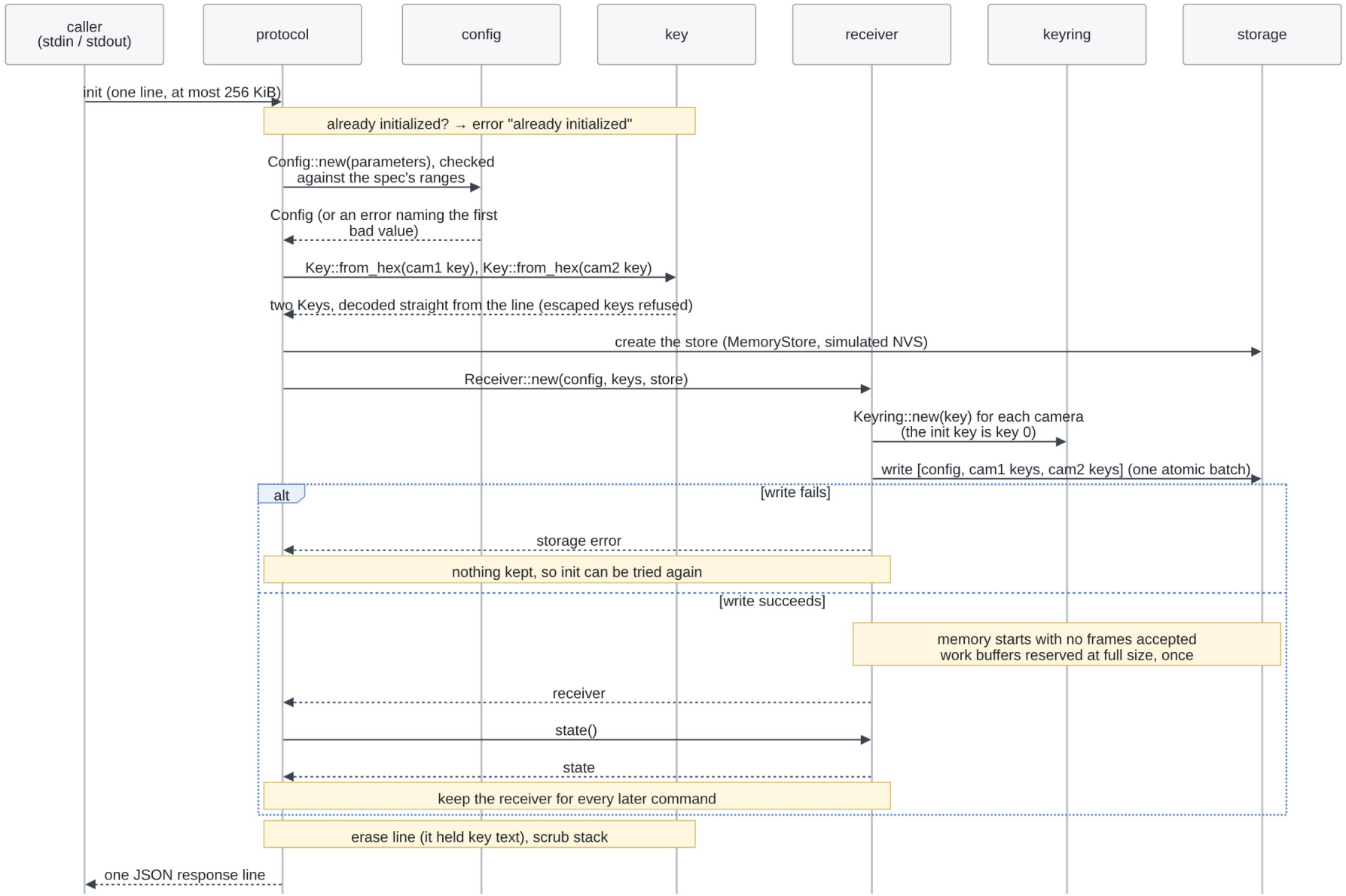
IMPLEMENTATION

Module map



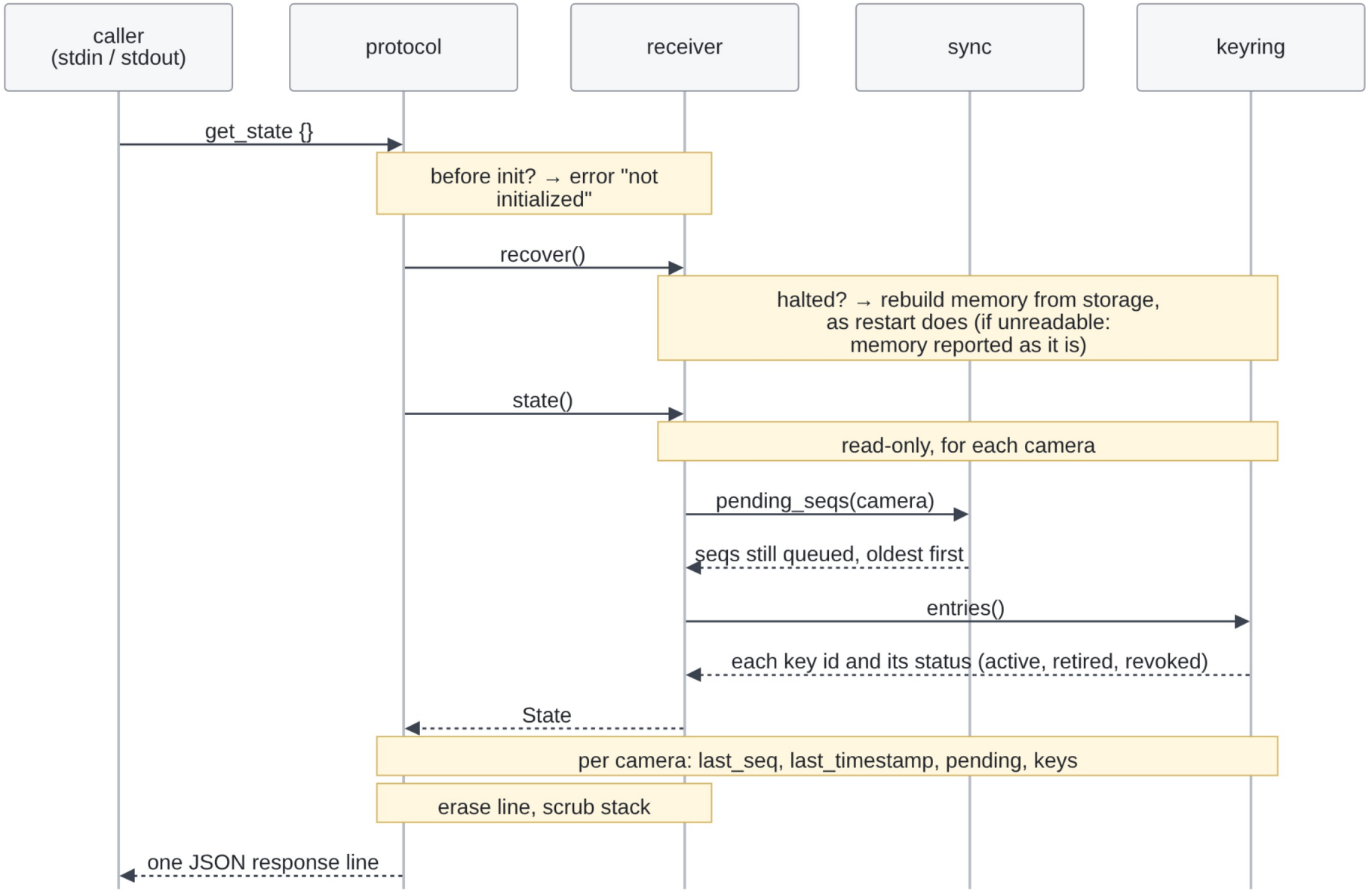
IMPLEMENTATION · SEQUENCE DIAGRAM

init



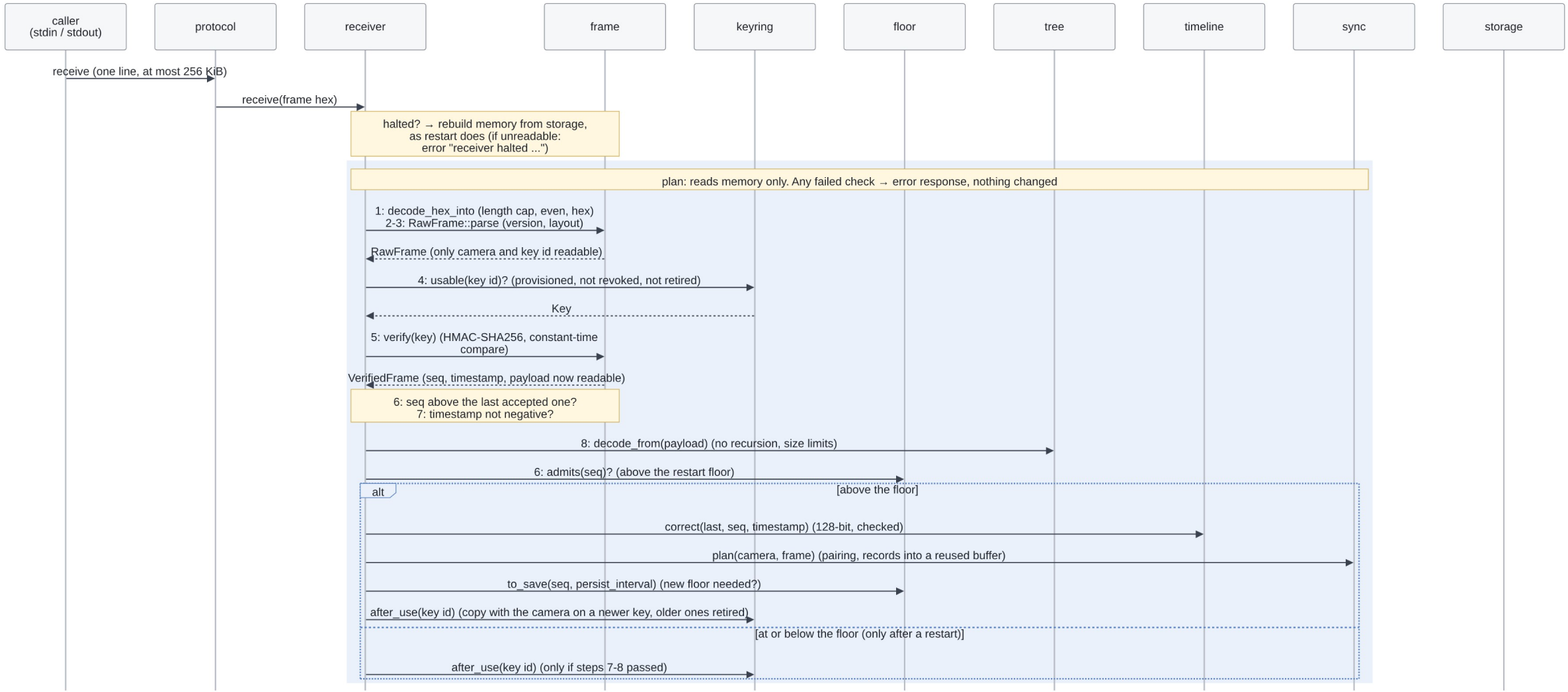
IMPLEMENTATION · SEQUENCE DIAGRAM

get_state



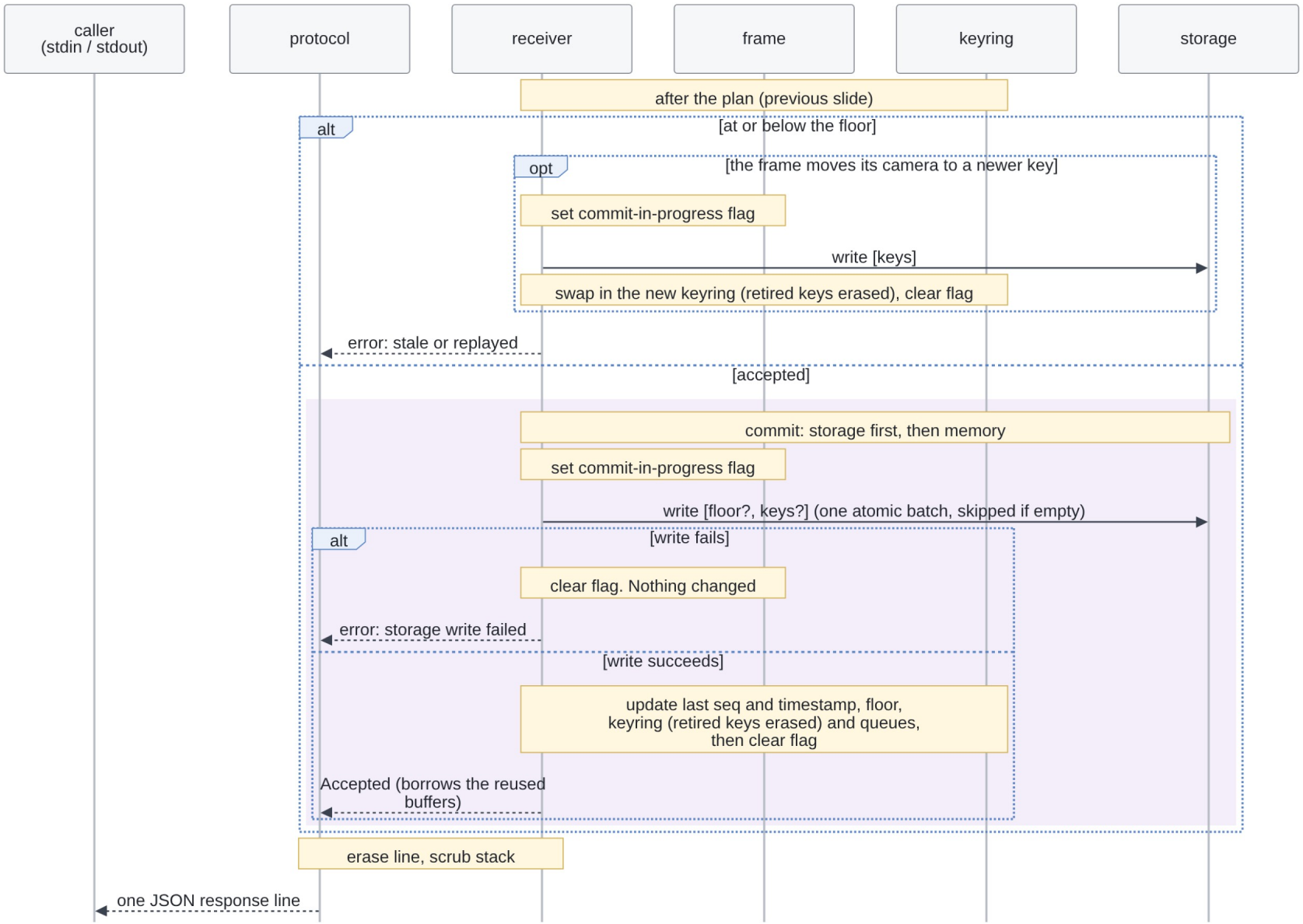
IMPLEMENTATION · SEQUENCE DIAGRAM

receive, part 1: plan



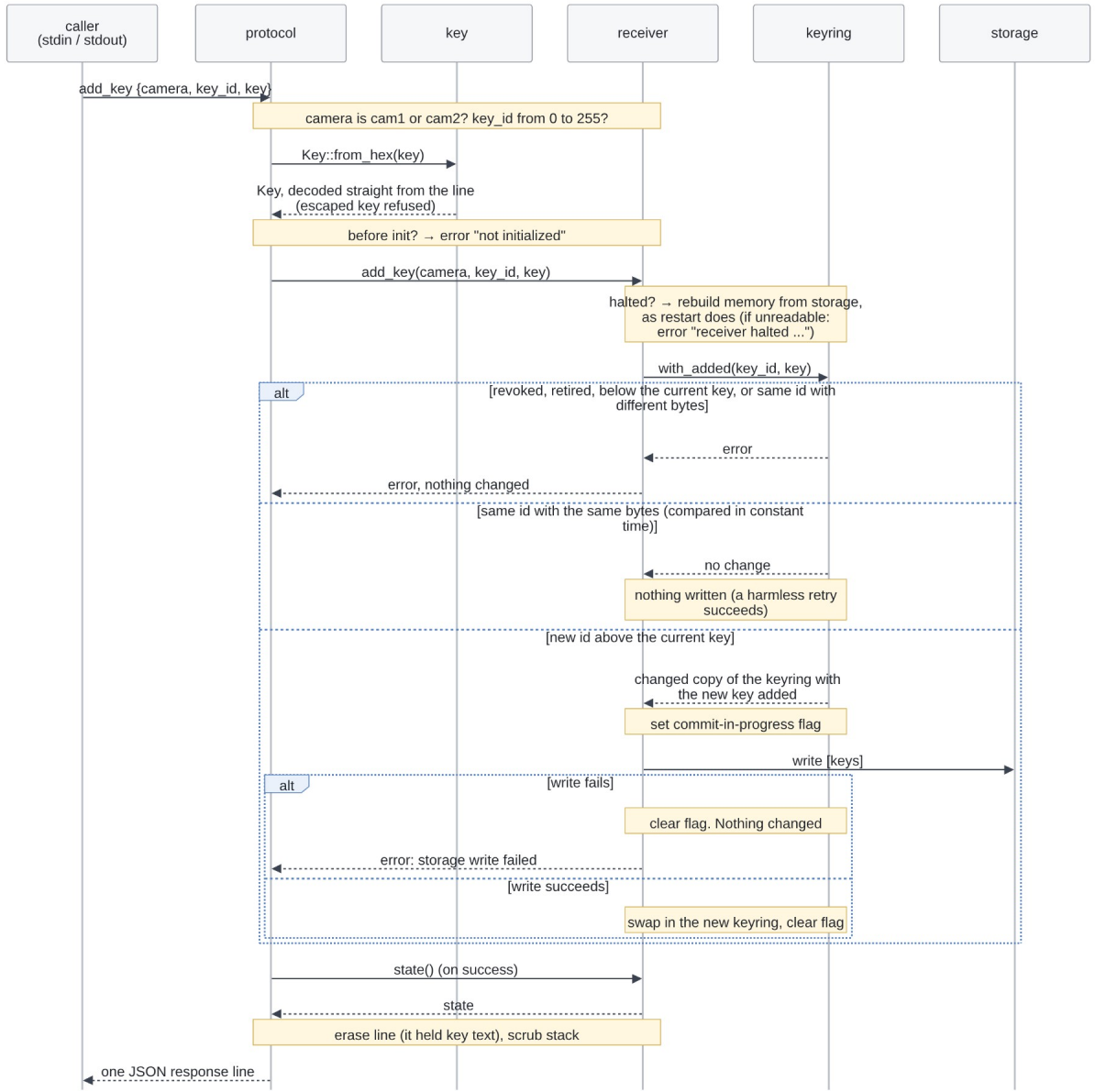
IMPLEMENTATION · SEQUENCE DIAGRAM

receive, part 2: commit



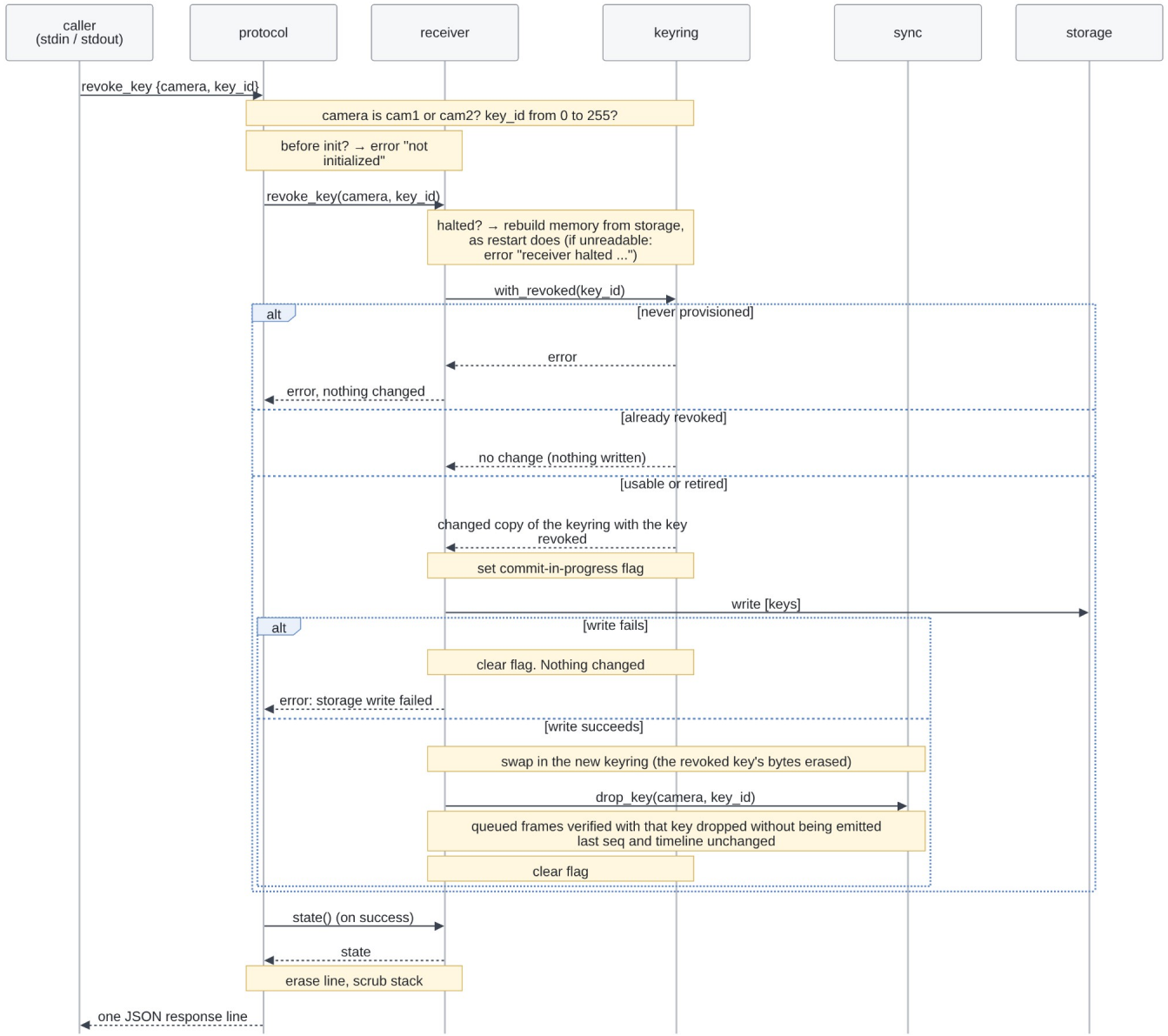
IMPLEMENTATION · SEQUENCE DIAGRAM

add_key

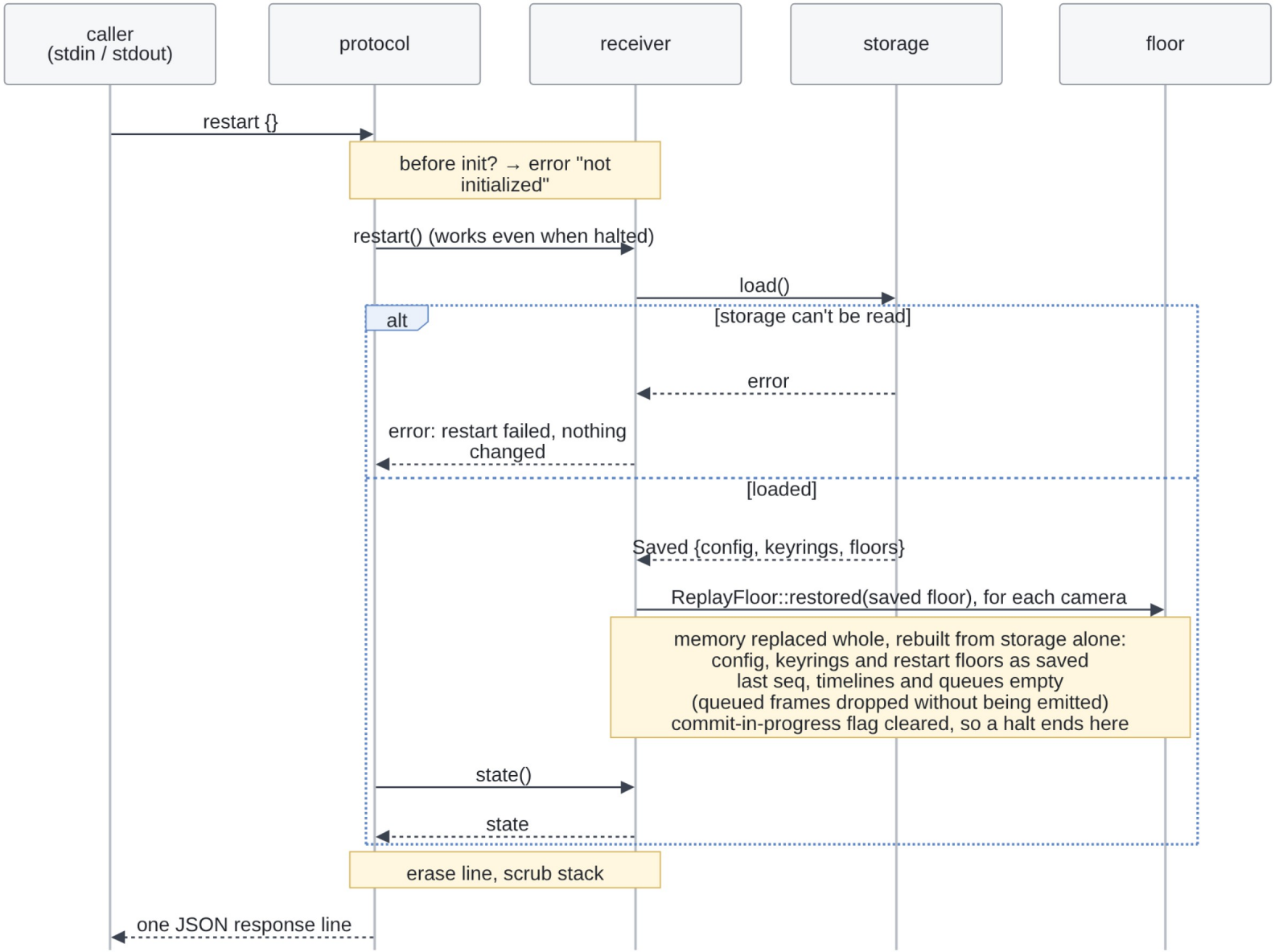


IMPLEMENTATION · SEQUENCE DIAGRAM

revoke_key



restart


caller
(stdin / stdout)
protocol
receiver
storage
floor
restart {}
before init? → error "not initialized"
restart() (works even when halted)
load()
alt
[storage can't be read]
error
error: restart failed, nothing changed
[loaded]
Saved {config, keyrings, floors}
ReplayFloor::restored(saved floor), for each camera
memory replaced whole, rebuilt from storage alone:
config, keyrings and restart floors as saved
last seq, timelines and queues empty
(queued frames dropped without being emitted)
commit-in-progress flag cleared, so a halt ends here
state()
state
erase line, scrub stack
one JSON response line

LIVE DEMO · ABOUT 5 MINUTES

demo/run.sh

Part 1

Normal traffic

Frames authenticated and paired; a glitched timestamp of 9999 corrected to 1055

Part 2

Attacks on the link

A replay, a modified frame, a line that isn’t JSON, a key in the wrong field: all rejected, nothing echoed

Part 3

Key rotation, revocation

Key 1 retires key 0; a frame forged with the old key is refused; a revoked key’s frames are refused

Part 4

Power loss

After a restart, a recorded frame is replayed and refused; seq 107 is accepted and paired

The transcript test checks every response in this session, so the demo can’t drift from the code.

TEST COVERAGE

The evidence in numbers

99.33%

line coverage; only three lines never run, and none is a real gap

220 of 220

mutants that compile are caught; the other 80 of 300 don’t compile

100,000

cases each for five property tests, all passing

10 min

per fuzz target (tree, command_line, receiver), with no failures

300,000

soak commands, with memory flat at 4 MB on macOS

TEST COVERAGE

What runs in CI, and what runs on request

Run in CI, on every push

- Unit tests
- Spec examples
- Rules and decisions
- Key rotation and revocation
- Restarts
- Reference model
- Safety net
- End-to-end
- Memory scan (Linux only)
- Soak
- Fuzzing, 60 seconds per target
- Coverage
- Dependencies

Run on request

- Throughput benchmark
- Mutation testing
- Longer fuzzing
- More property test cases
- Soak stress run
- Detailed coverage

PERFORMANCE

Throughput and memory

Requirement: the spec's worst case is two cameras at 1 ms, which is 2,000 frames per second. Even the largest possible frames keep up.

Frames	Size	Frames per second	µs per frame
Small (one-node tree)	55 bytes	41,000–43,000	23–24
Largest accepted (1,024-node tree)	6,193 bytes	11,000–15,000	68–89
Largest possible (tag checked, tree refused)	65,583 bytes	2,200–3,100	320–450

~4 MB

resident on Mac, flat over 300,000 commands

656 KiB

peak heap at max_pending 1,024, mostly reserved once

FUTURE ENHANCEMENTS

Three limits worth fixing next

The sequence number runs out

At the spec's minimum period_ms of 1, the 32-bit seq runs out in about 49.7 days (6.8 years at 50 ms). **Fix:** a new frame version with 64-bit sequence numbers.

A stolen key can lock a camera out

It can push seq to its maximum, and there's no way to re-provision the receiver. **Fix:** sequence numbering per key, or a re-provisioning command that resets a camera's state.

A wide pair window is ambiguous

The spec allows pair_window_ms up to period_ms − 1, where one dropped frame can shift every later pair. **Fix:** require 2 × pair_window_ms < period_ms − max_jitter_ms.

FUTURE ENHANCEMENTS

Before a real deployment

- Defend against attackers with physical access to the node
- Check freshness: camera time against the node’s clock
- Recover a camera whose sequence counter restarts
- Count authentication failures, and alert on them

AI USAGE

Claude wrote the code; the decisions were mine

Claude wrote

The code, tests, scripts, README.md, AGENTS.md, CLAUDE.md and this presentation.

I decided and reviewed

I made or approved all the design decisions, and reviewed everything Claude produced.

I wrote DESIGN.md

Written manually, with AI used only for grammar, research and feedback.

THANK YOU

Questions?

`README.md`
Build, run and test

`DESIGN.md`
The threat model and every decision

`demo/run.sh`
The demo, step by step

`github.com/andychan1998/secure-sensor-link`